

Programação Funcional: do Cálculo Lambda à Implementação de Métodos Numéricos

Enzo Rocha Leite Diniz Ribas Luis A. D'Afonseca Lucas Martins Rocha

Centro Federal de Educação Tecnológica de Minas Gerais
Belo Horizonte – MG

Semana de Iniciação Científica – CEFET-MG

- 1 Motivação
- 2 Programação Imperativa
- 3 Programação Funcional
- 4 Cálculo Lambda
- 5 Currying e Composição
- 6 Monads e Efeitos
- 7 Métodos Numéricos
- 8 Vantagens e Limitações
- 9 Conclusão

O que significa implementar uma função matemática?

- Na matemática, uma função relaciona entradas válidas a saídas.
- Na programação, uma função pode modificar estado, realizar E/S ou produzir erros.
- A programação funcional busca aproximar a computação do comportamento matemático das funções.

Ideia-guia

Quanto mais explícito for o comportamento de uma função, mais fácil é raciocinar, testar e compor o programa.

Do problema matemático ao programa

Considere

$$f(x) = \sqrt{x}, \quad \text{Dom}(f) = \{x \in \mathbb{R} : x \geq 0\}.$$

- $f(4) = 2$
- $f(0) = 0$
- $f(-1)$ não possui resultado em $\mathbb{R}$

Problema computacional

Como representar, no programa, que nem toda entrada possui um resultado válido?

Uma implementação imperativa

```
double sqrt(double x) {  
    if (x < 0)  
        throw new IllegalArgumentException();  
  
    return Math.sqrt(x);  
}
```

- comportamento normal;
- condição de erro;
- exceção durante a execução.

Exceções: uma solução, mas com custo conceitual

- Exceções são importantes para representar falhas.
- O fluxo de erro fica separado do fluxo normal.
- A assinatura tradicional nem sempre comunica as falhas possíveis.

Pergunta

E se o possível erro fizesse parte do próprio resultado?

$$\text{sqrt} : \mathbb{R} \longrightarrow \text{Maybe } \mathbb{R}$$

Entrada	Comportamento esperado
4	2
0	0
-1	falha controlada

- Testes verificam comportamentos específicos.
- Tipos e contratos podem tornar erros explícitos.
- A programação funcional explora essa ideia sistematicamente.

O que é programação funcional?

A programação funcional trata **funções como abstrações fundamentais da computação**.

- funções de primeira classe;
- funções puras;
- imutabilidade;
- composição;
- recursão;
- efeitos colaterais controlados ou isolados.

Função pura

Para uma mesma entrada, uma função pura produz sempre a mesma saída e não altera o estado observável do programa.

Função matemática × função pura

$$f(x) = x^2 + 1$$

Uma função pura pode ser vista como

$$f : \mathbb{R} \rightarrow \mathbb{R}.$$

- Não depende de variável global.
- Não altera variável externa.
- Não escreve em arquivo.
- Não depende de entrada externa.

Resultado

A função pode ser analisada como uma transformação entrada $\rightarrow$ saída.

```
int counter = 0;

int f(int x) {
    counter++;
    return x * x;
}
```

Embora pareça simples, o comportamento observável depende de estado externo.

Consequência

A função deixa de ser apenas uma transformação entre entrada e saída.

A base formal: Cálculo Lambda

O cálculo lambda fornece um modelo formal para representar funções e computações.

- ① **Abstração:** definir uma função;
- ② **Aplicação:** aplicar uma função;
- ③ **Substituição:** avaliar a aplicação.

Exemplo:

$$\lambda x. x^2$$

representa uma função que recebe x e retorna x^2 .

BNF: descrevendo a sintaxe

Uma gramática simples:

$$\begin{aligned}\langle expr \rangle ::= & \langle var \rangle \\ & | \lambda \langle var \rangle . \langle expr \rangle \\ & | \langle expr \rangle \langle expr \rangle\end{aligned}$$

- $\lambda x.e$: abstração;
- $e_1 e_2$: aplicação;
- $\langle var \rangle$: variável.

Ponte conceitual

O cálculo lambda conecta a definição matemática de função a uma representação formal de computação.

Considere

$$(\lambda x. x * x) 5.$$

Pela substituição $x \leftarrow 5$:

$$5 * 5 = 25.$$

A redução beta é

$$(\lambda x. e) v \longrightarrow e[x := v].$$

Ideia

Programar pode ser interpretado como construir e reduzir expressões.

Currying

Uma função pode ser escrita como

$$f(x, y) = x + y.$$

No cálculo lambda:

$$f = \lambda x. \lambda y. x + y.$$

Aplicação:

$$f(2)(3) = 5.$$

Currying

Uma função de vários argumentos é representada como uma sequência de funções de um único argumento.

Por que Currying é útil?

`somar x y = x + y.`

Fixando $x = 10$:

`somar10 = somar 10.`

Logo,

`somar10(5) = 15.`

- especialização;
- reutilização;
- composição;
- construção de pipelines.

Do erro para um valor

Em vez de

$\text{sqrt}(x) \rightarrow \text{exceção},$

podemos modelar

$\text{sqrt}(x) \rightarrow \text{Maybe } \mathbb{R}.$

$$\begin{cases} \text{Just}(y), & x \geq 0, \\ \text{Nothing}, & x < 0. \end{cases}$$

O possível fracasso passa a fazer parte do resultado.

Monads: composição com contexto

Uma apresentação inicial de Monad:

uma abstração para compor computações que carregam um contexto.

Exemplos:

- ausência: Maybe;
- erro: Either;
- entrada e saída: IO;
- estado;
- listas.

Cuidado conceitual

Monad não é simplesmente “tratamento de exceções”. O ponto central é a composição de computações contextualizadas.

Compondo operações que podem falhar

Considere

$$x \xrightarrow{\sqrt{}} \sqrt{x} \xrightarrow{\log} \log(\sqrt{x}).$$

Ambas possuem restrições de domínio. Com um contexto como Maybe, podemos compor:

$$\text{Maybe } A \longrightarrow (A \longrightarrow \text{Maybe } B) \longrightarrow \text{Maybe } B.$$

Se uma etapa falhar, a composição preserva o contexto de falha.

Por que isso importa para métodos numéricos?

Métodos numéricos são algoritmos construídos a partir de transformações matemáticas. Para Newton–Raphson:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

Definindo

$$N_f(x) = x - \frac{f(x)}{f'(x)},$$

temos

$$x_{n+1} = N_f(x_n).$$

Perspectiva funcional

A iteração pode ser vista como aplicação repetida de uma função.

Newton-Raphson: formulação funcional

$$N(f, f')(x) = x - \frac{f(x)}{f'(x)}.$$

A iteração:

$$x_0, \quad N(x_0), \quad N^2(x_0), \quad N^3(x_0), \dots$$

ou

$$x_{n+1} = N(x_n).$$

- f é uma função recebida como argumento;
- f' é uma função recebida como argumento;
- N constrói a próxima aproximação.

Newton-Raphson: versão imperativa

```
x = x0;  
  
for (int i = 0; i < maxIter; i++) {  
    x = x - f(x) / df(x);  
  
    if (abs(f(x)) < tol)  
        break;  
}
```

O estado é atualizado:

$$x^{(0)} \rightarrow x^{(1)} \rightarrow x^{(2)} \rightarrow \dots$$

Contraste

A implementação é descrita como uma sequência de mutações de estado.

Newton-Raphson: perspectiva funcional

```
newtonStep f df x =  
  x - f x / df x
```

Sua estrutura é:

$$(f, f', x) \mapsto x - \frac{f(x)}{f'(x)}.$$

Uma iteração completa pode ser construída pela aplicação repetida de `newtonStep`.

Observação

A estrutura matemática da iteração fica explícita no programa.

Casos problemáticos em métodos numéricos

Métodos numéricos possuem condições de falha:

- derivada nula;
- divergência;
- máximo de iterações;
- domínio inválido;
- perda de precisão numérica.

Uma representação explícita pode ser:

```
NewtonResult = Converged | MaxIterations | InvalidDerivative | ...
```

Assim, o algoritmo não precisa representar todos os resultados como um simples número.

Funções puras

- previsibilidade;
- testes;
- raciocínio matemático.

Imutabilidade

- menos estado compartilhado;
- menos dependências ocultas.

Composição

- modularidade;
- pipelines.

Tipos e contextos

- erros explícitos;
- contratos mais claros.

Currying

- especialização;
- reutilização.

Paradigmas são ferramentas, não religiões.

Linguagens modernas combinam paradigmas:

- Java: lambdas e streams;
- Python: funções de primeira classe;
- JavaScript: composição e funções de alta ordem;
- Rust: imutabilidade, pattern matching e tipos algébricos;
- Haskell: forte orientação funcional.

Questão de engenharia

A escolha deve considerar clareza, segurança, desempenho, domínio do problema e custo de desenvolvimento.

- 1 Funções matemáticas fornecem uma base conceitual natural.
- 2 O imperativo descreve computação por mudanças de estado.
- 3 O funcional privilegia funções puras, imutabilidade e composição.
- 4 O cálculo lambda fornece uma fundamentação formal para funções e aplicação.
- 5 Currying permite especialização e composição.
- 6 Monads estruturam computações que carregam contexto.
- 7 Métodos numéricos podem ser expressos por composição e aplicação de funções.

Do cálculo à computação.

Funções matemáticas → Cálculo Lambda → Programação Funcional →
Métodos Numéricos

Obrigado.

-  PIERCE, B. C.
Types and Programming Languages.
MIT Press, 2002.
-  HUTTON, G.
Programming in Haskell.
Cambridge University Press.
-  BURDEN, R. L.; FAIRES, J. D.
Numerical Analysis.
Cengage Learning.
-  ABELSON, H.; SUSSMAN, G.
Structure and Interpretation of Computer Programs.
MIT Press.